



DESIGN AND ANALYSIS OF ALGORITHMS

Shylaja S S

Department of Computer Science
& Engineering

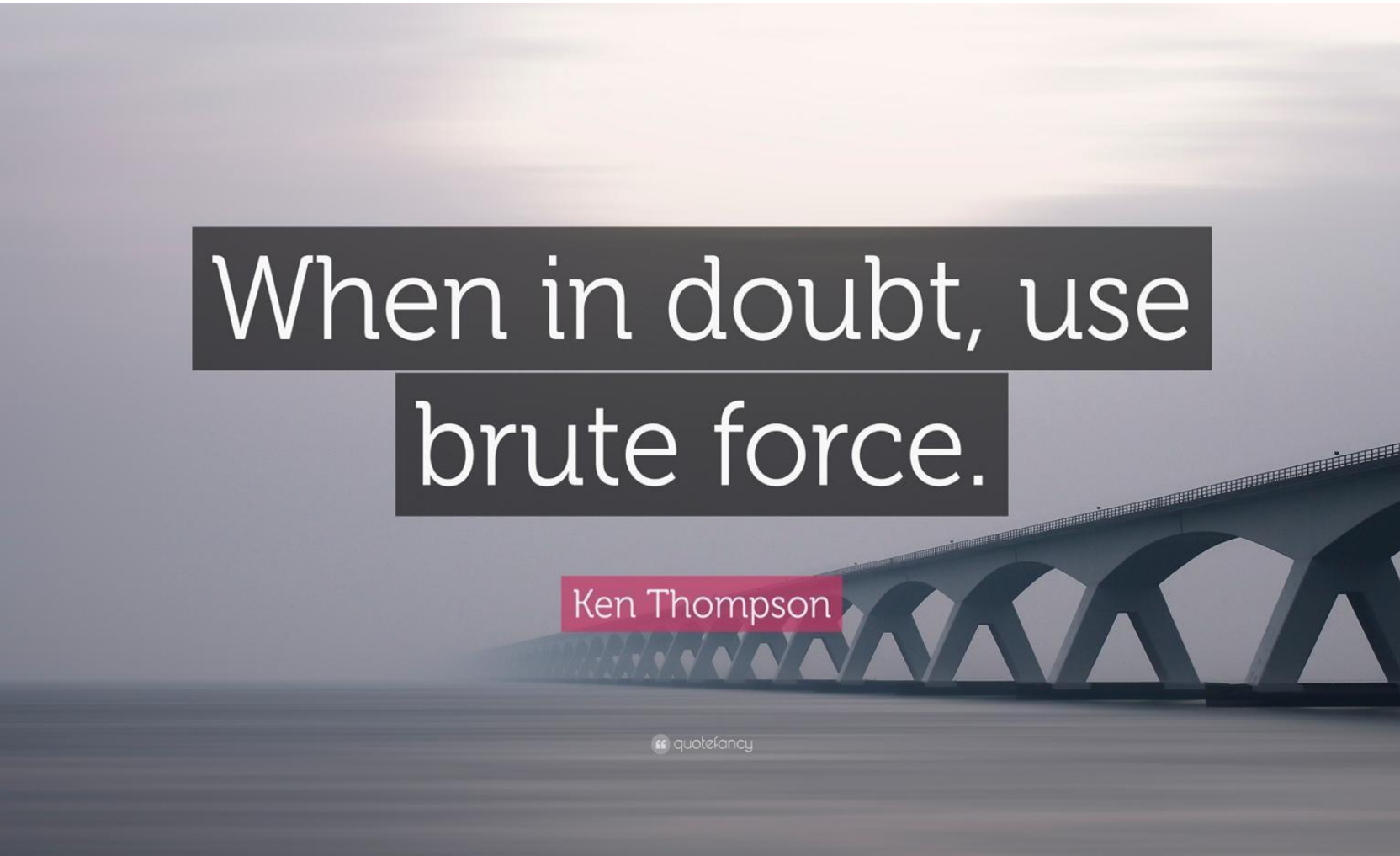
DESIGN AND ANALYSIS OF ALGORITHMS

Brute Force: Selection Sort and Bubble Sort

Major Slides Content: Anany Levitin

Shylaja S S

Department of Computer Science & Engineering

The background of the slide is a photograph of a long, multi-arched bridge spanning a body of water. The sky is overcast and grey, and the water is calm. The bridge's arches create a rhythmic pattern of light and dark shapes.

When in doubt, use
brute force.

Ken Thompson

- Brute Force is a straightforward approach to solving a problem, usually directly based on the problem statement and definitions of the concepts involved

Examples:

1. Searching for a key of a given value in a list
 2. Computing $n!$
 3. Computing a^n ($a > 0$, n a nonnegative integer)
 4. Hacking a password by matching all possible passwords.
- “Force” by the computer in terms of effort, but simple in strategy to implement.
 - Trial and Error method of trying out in some order.
 - Exhaustive effort rather than employing intellectual strategies.
 - Often one of the easiest way to solve it.

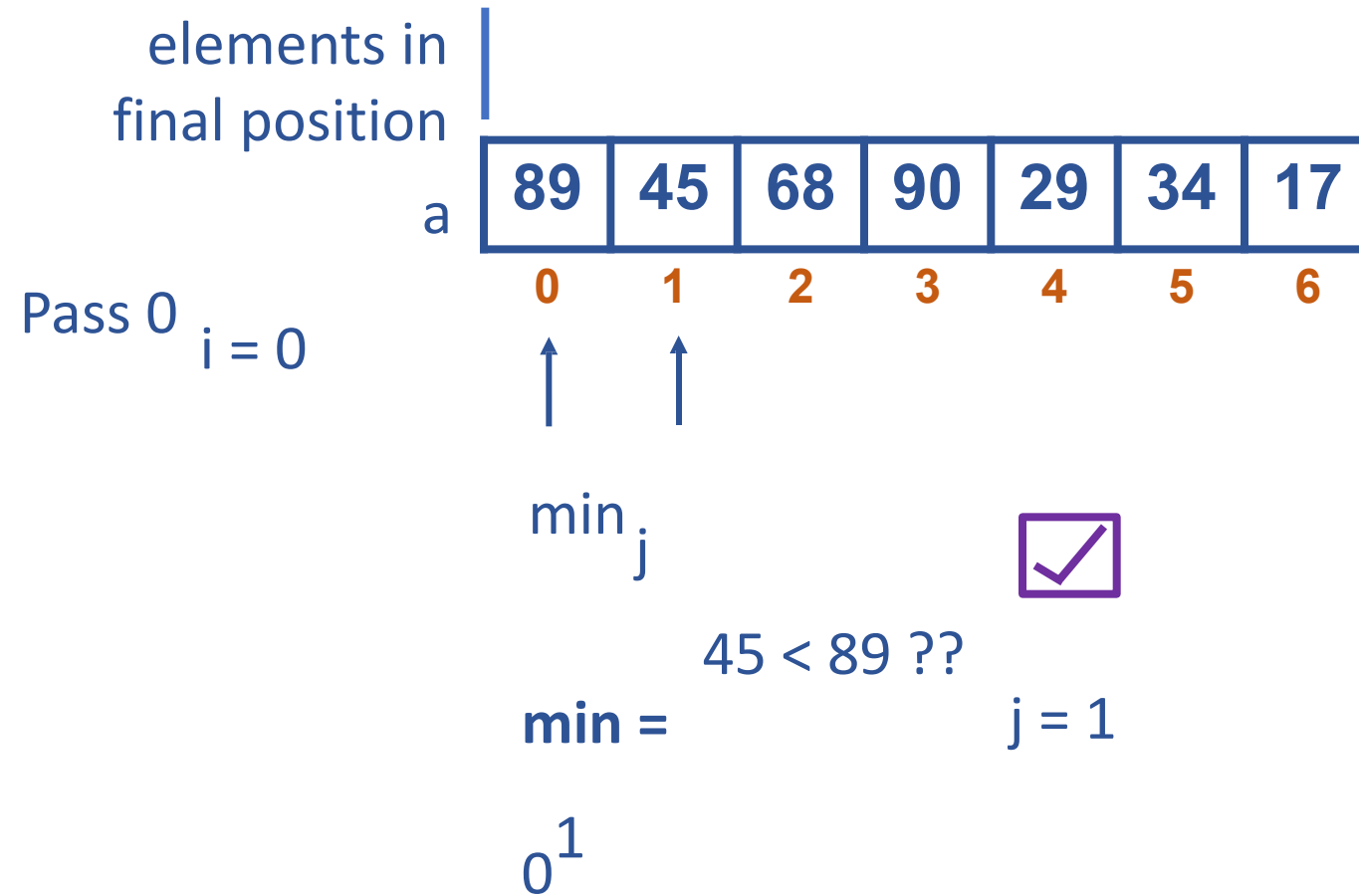
- Selection Sort
- Bubble Sort

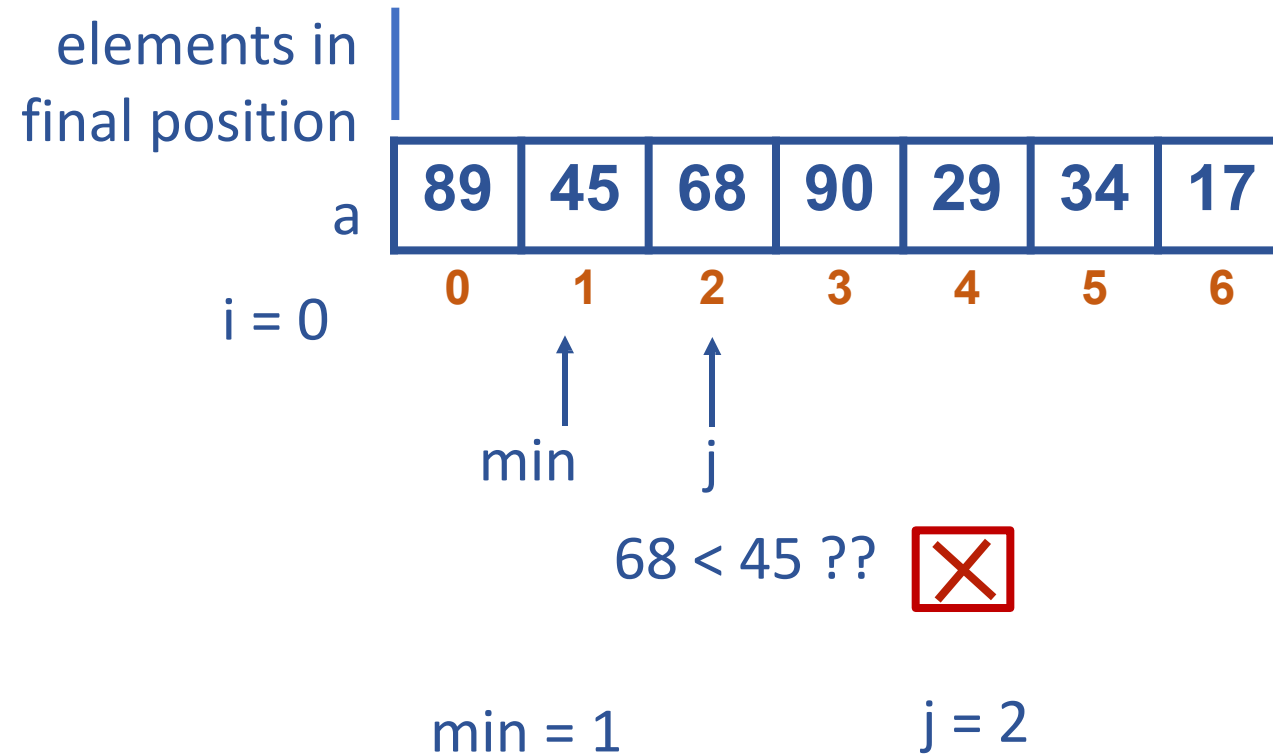
Selection Sort

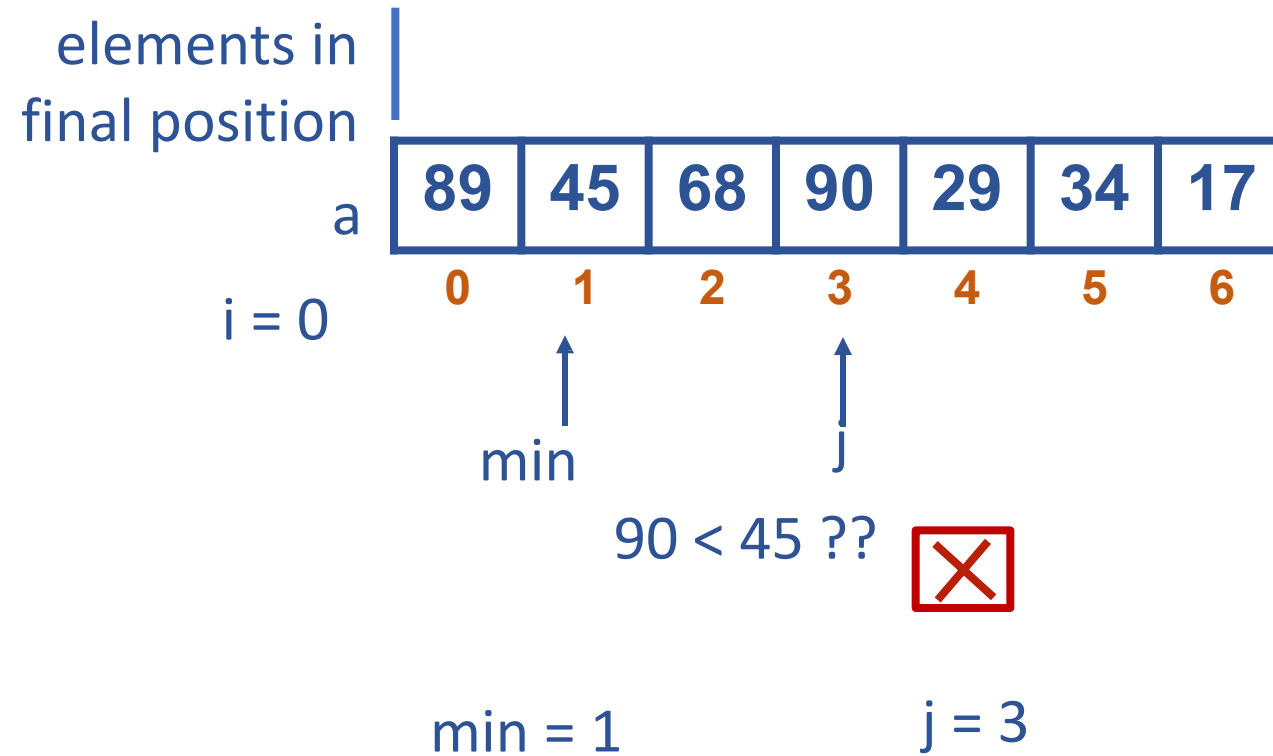
- Scan the array to find its smallest element and swap it with the first element, putting the smallest element in its final position in the sorted list
- Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second element, putting the second smallest element in its final position
- Generally, on pass i ($0 \leq i \leq n-2$), find the smallest element in $A[i..n-1]$ and swap it with $A[i]$

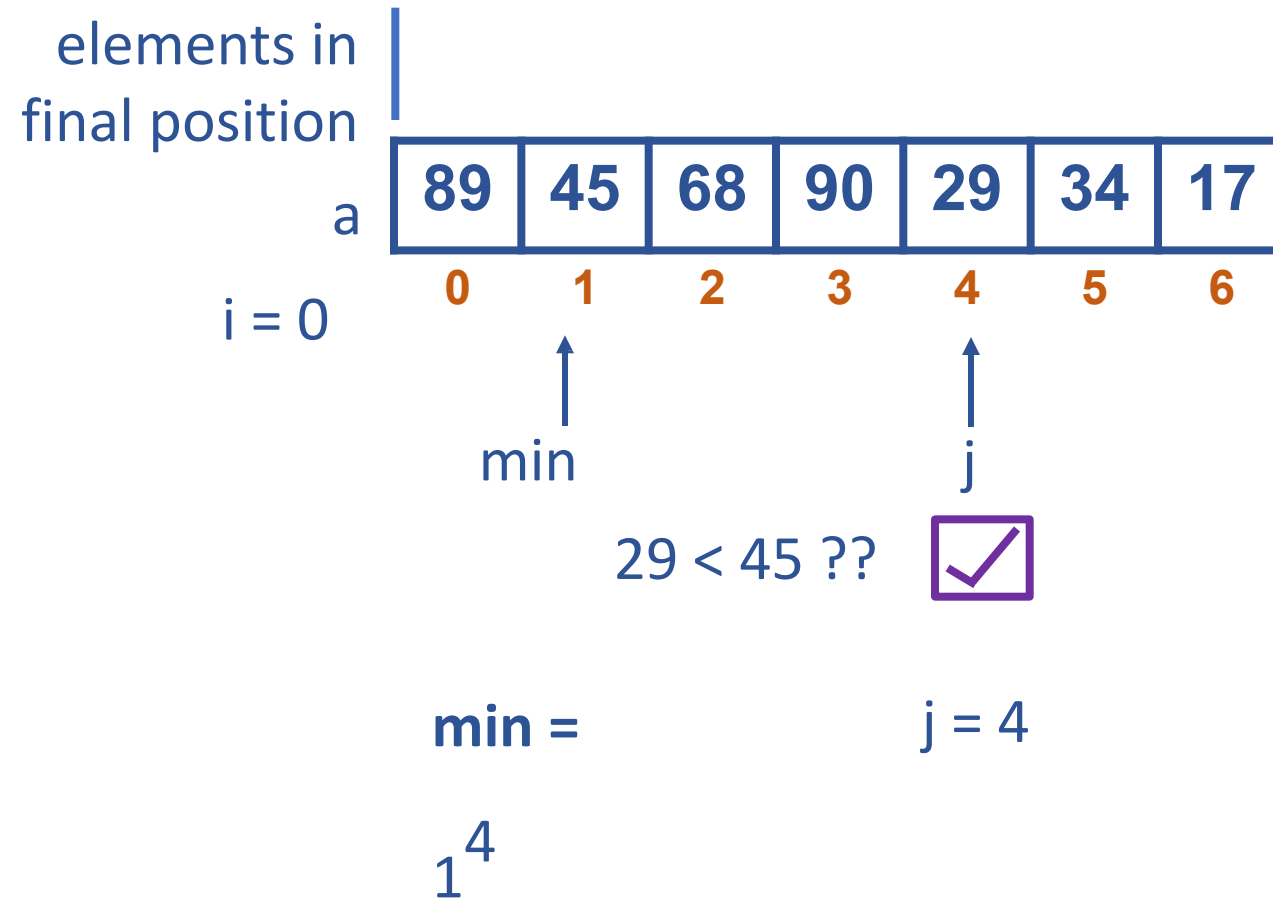


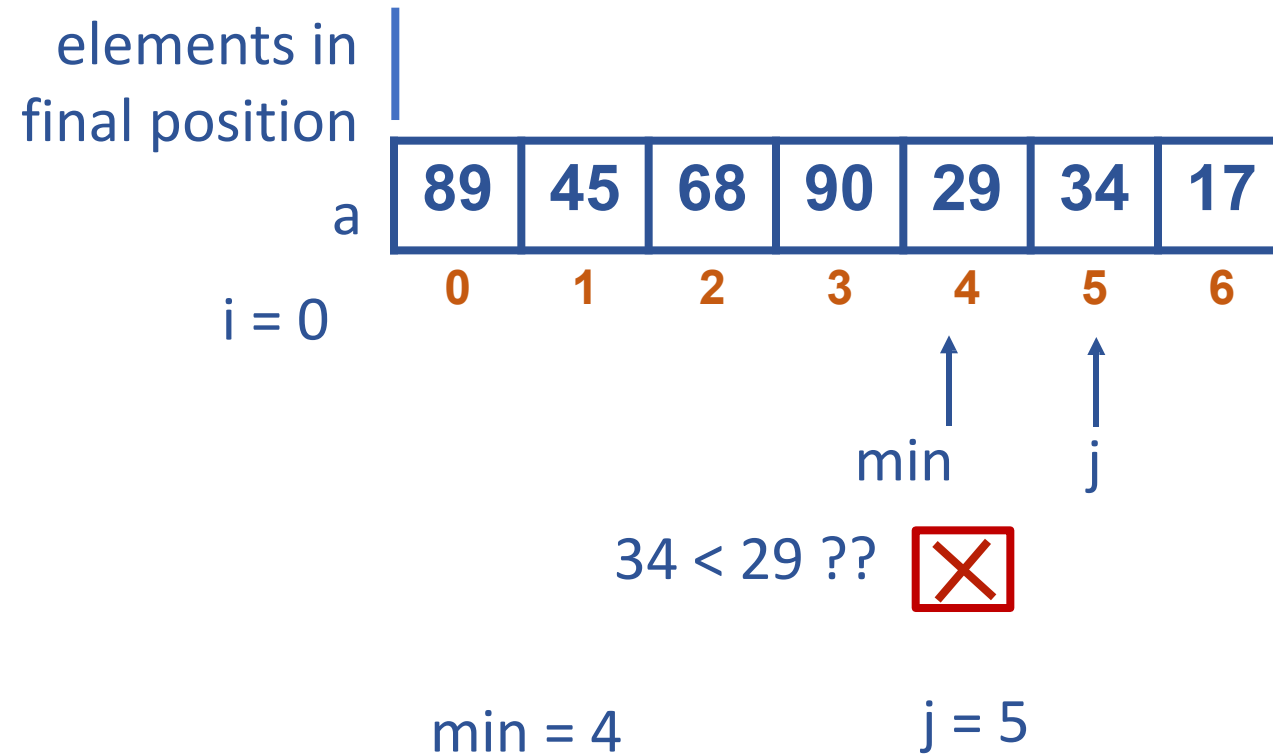
- $A[0] \leq A[1] \leq A[2] \dots \leq A[i-1] \mid A[i], \dots, A[\text{min}], \dots, A[n-1]$
- in their final positions the last $n - i$ elements

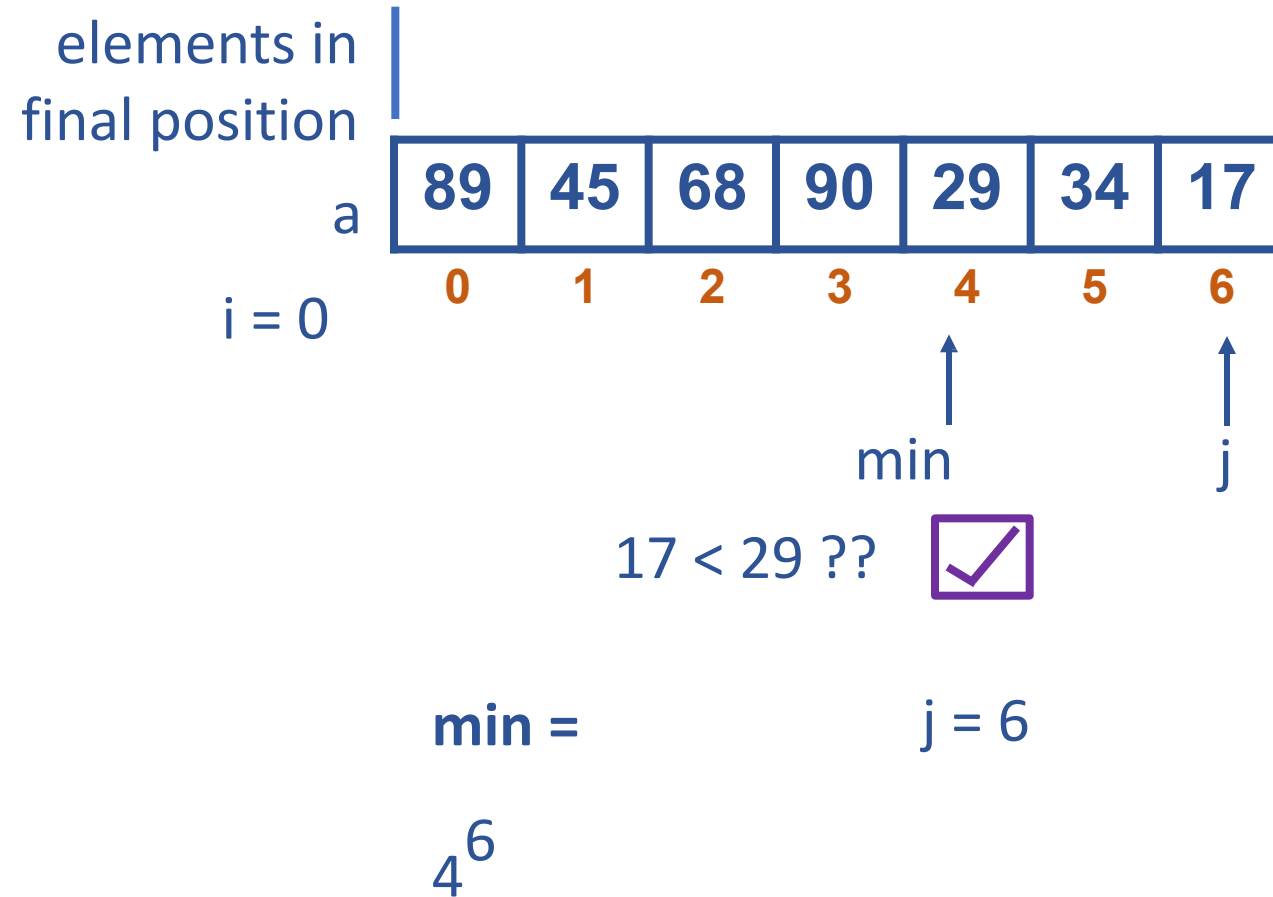


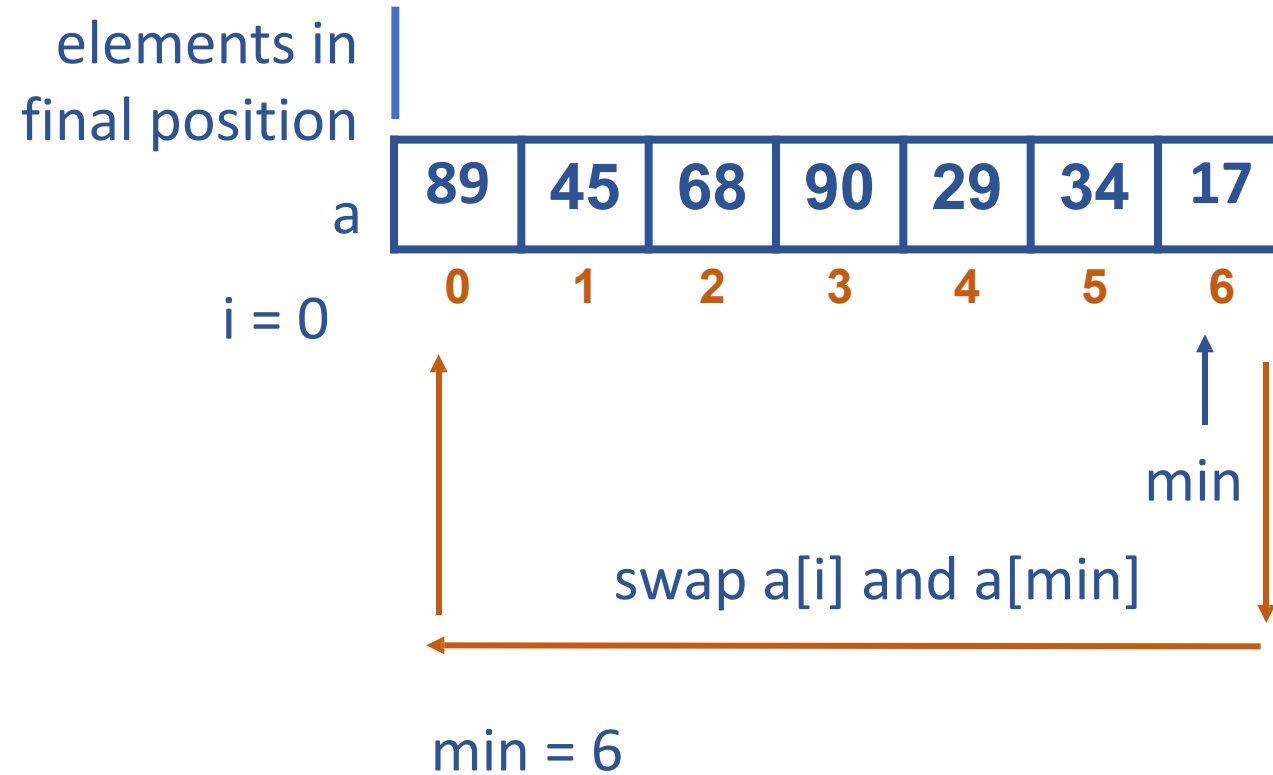


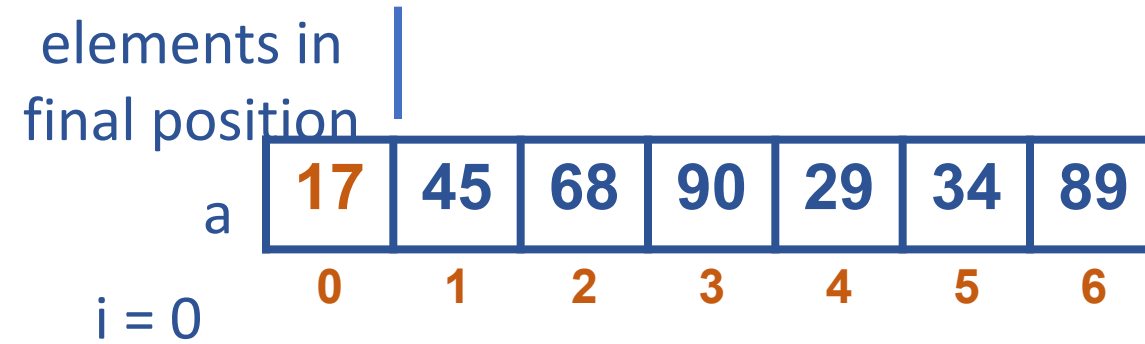


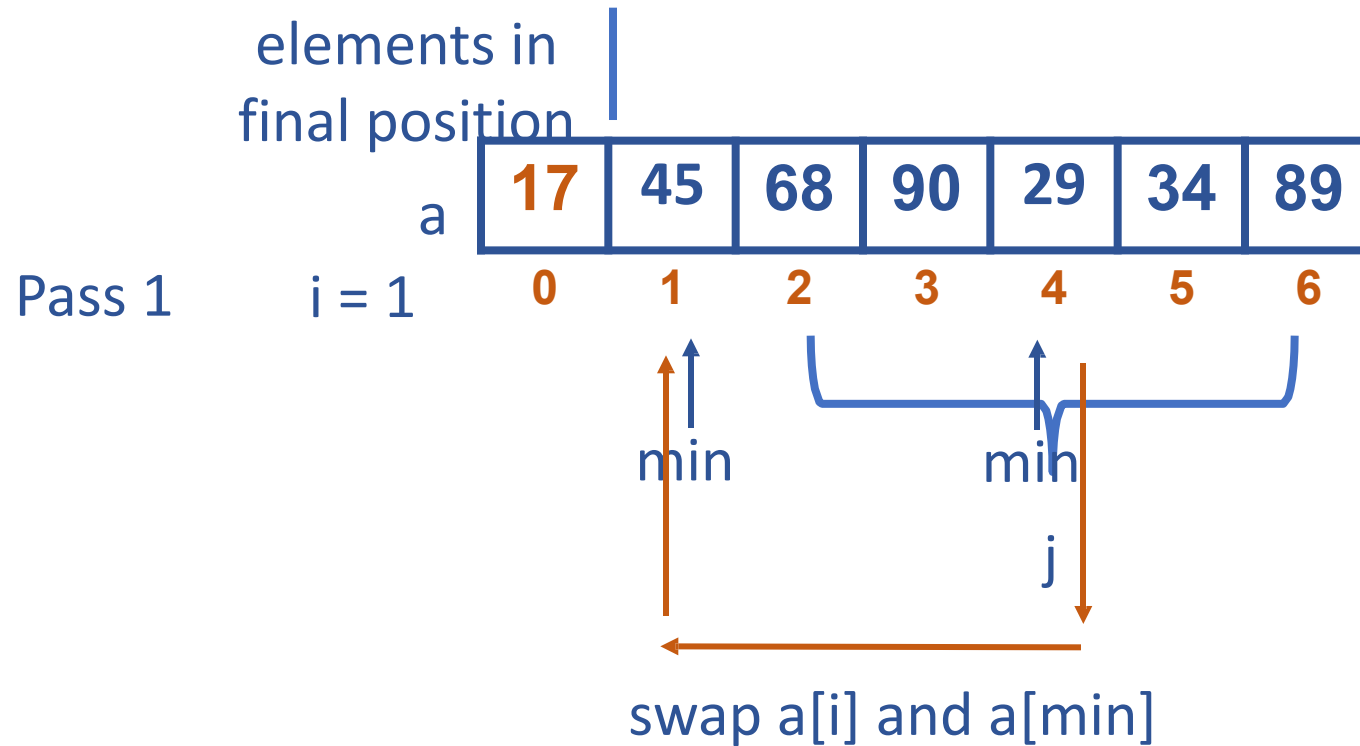




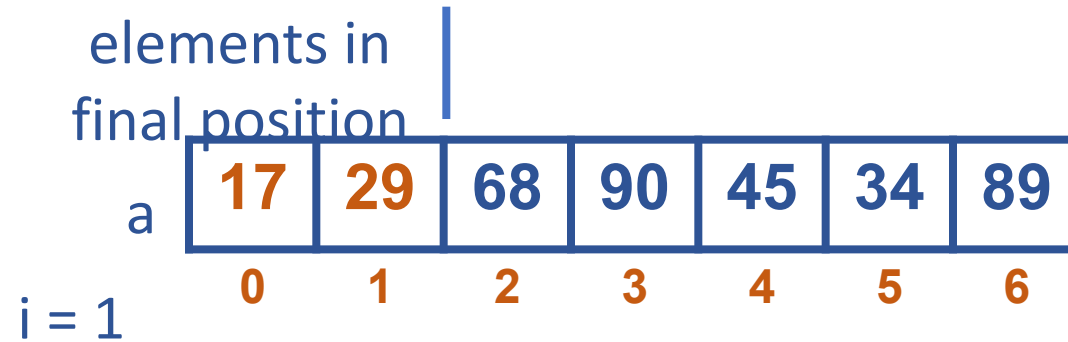


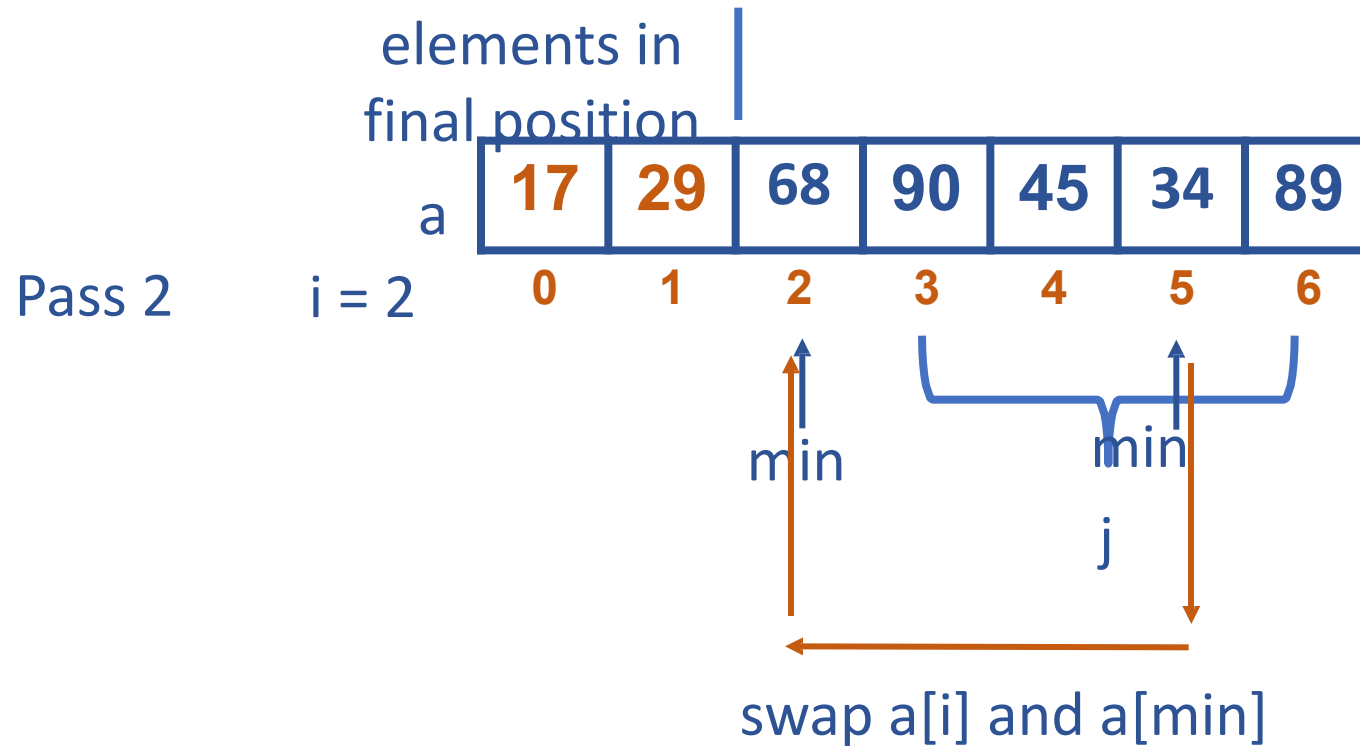




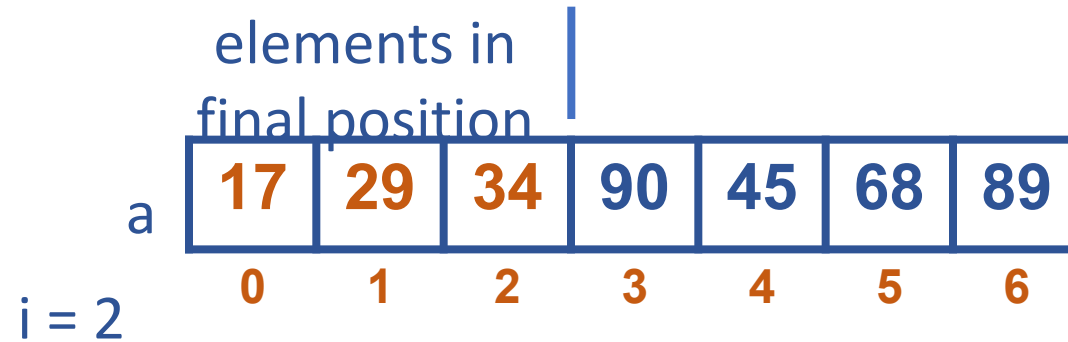


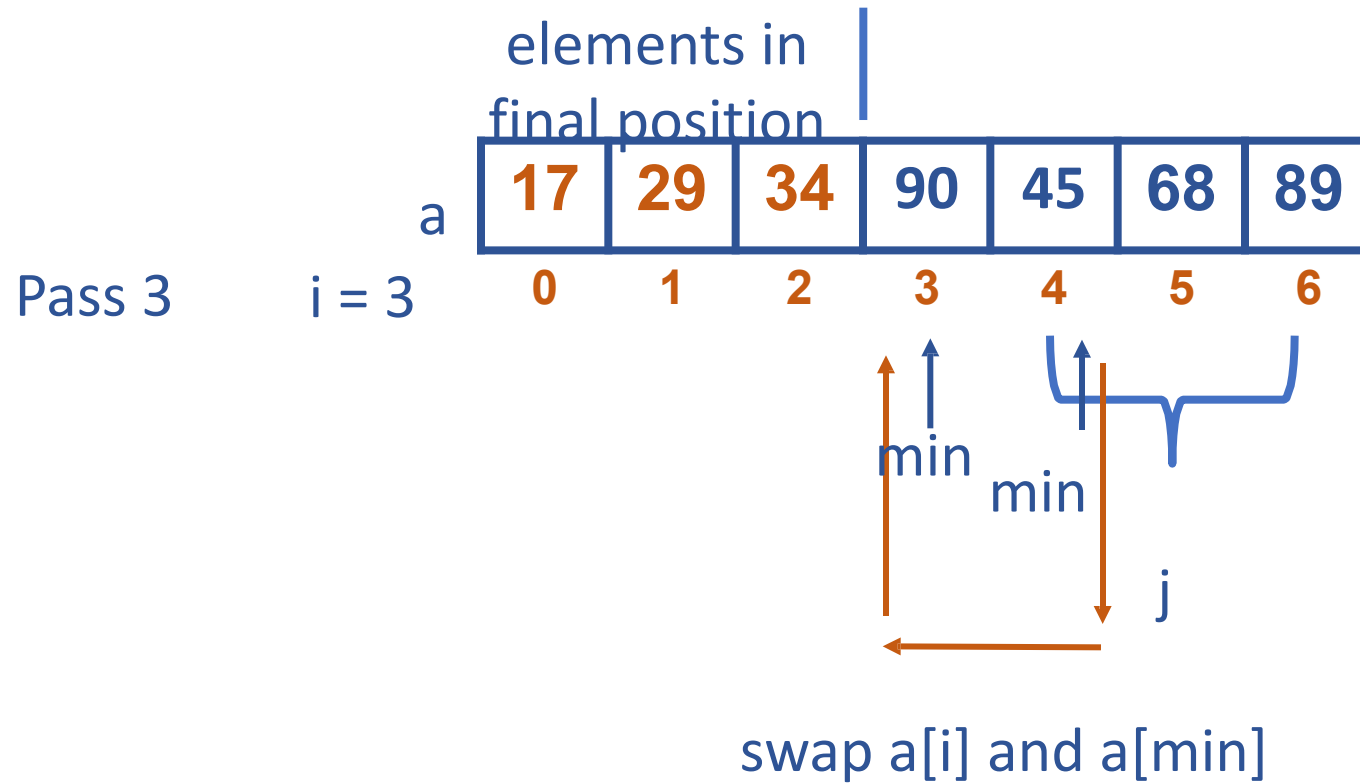
$\text{min} = 4$



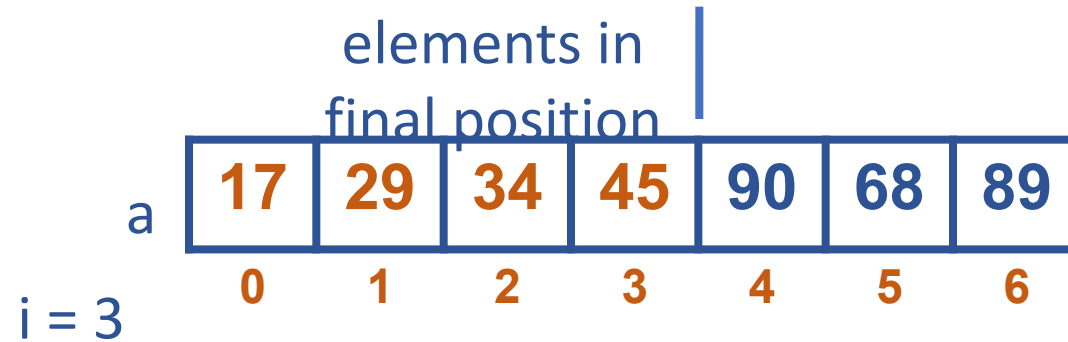


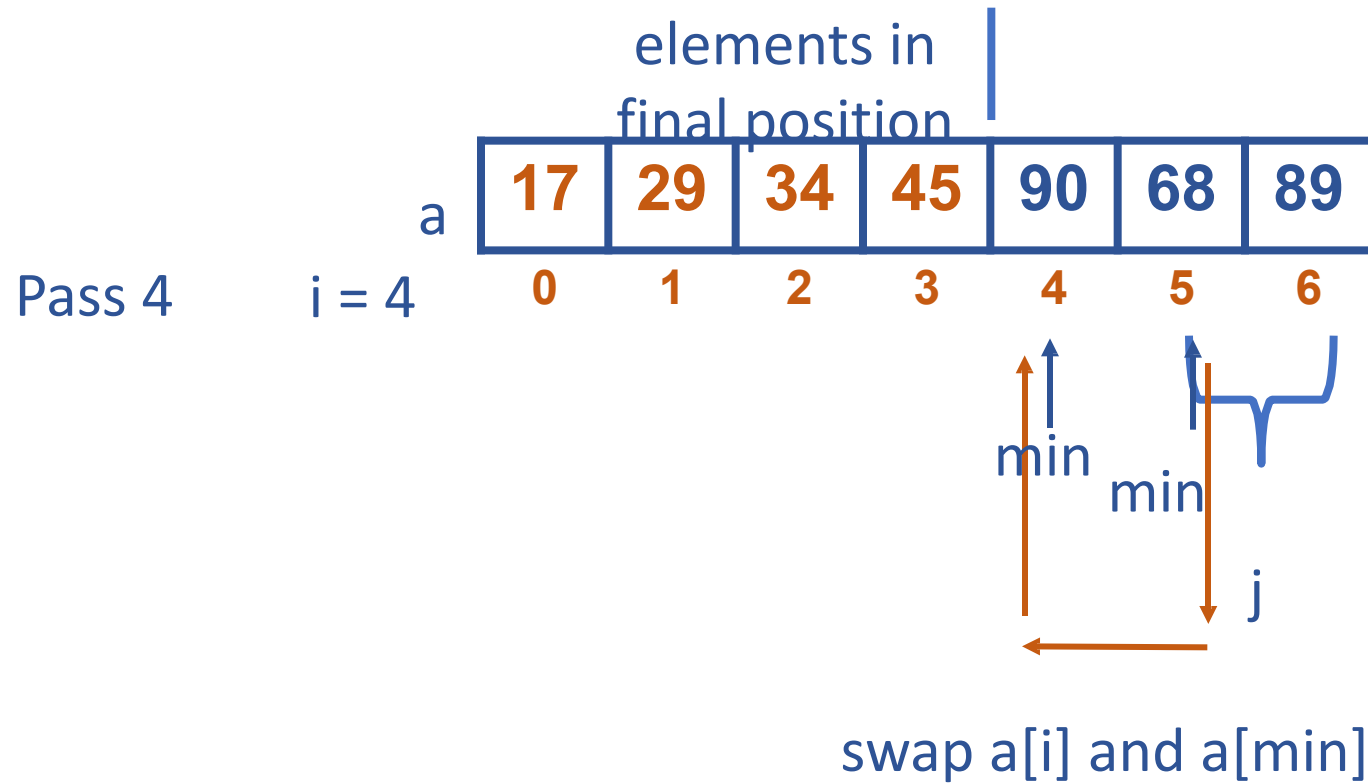
min = 5



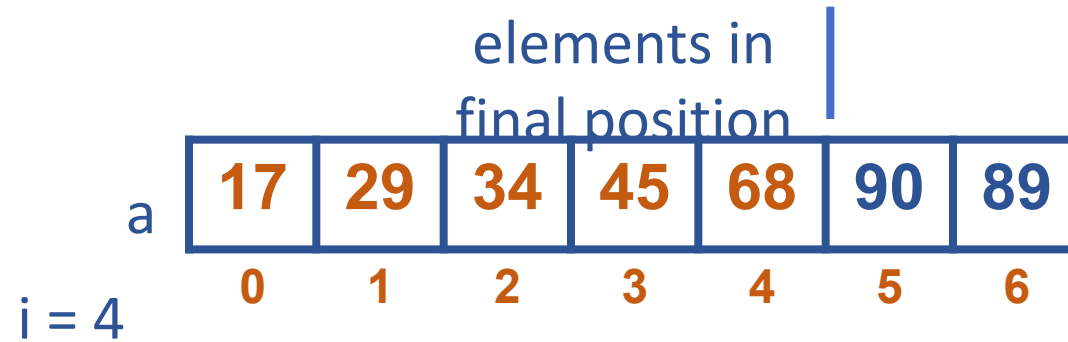


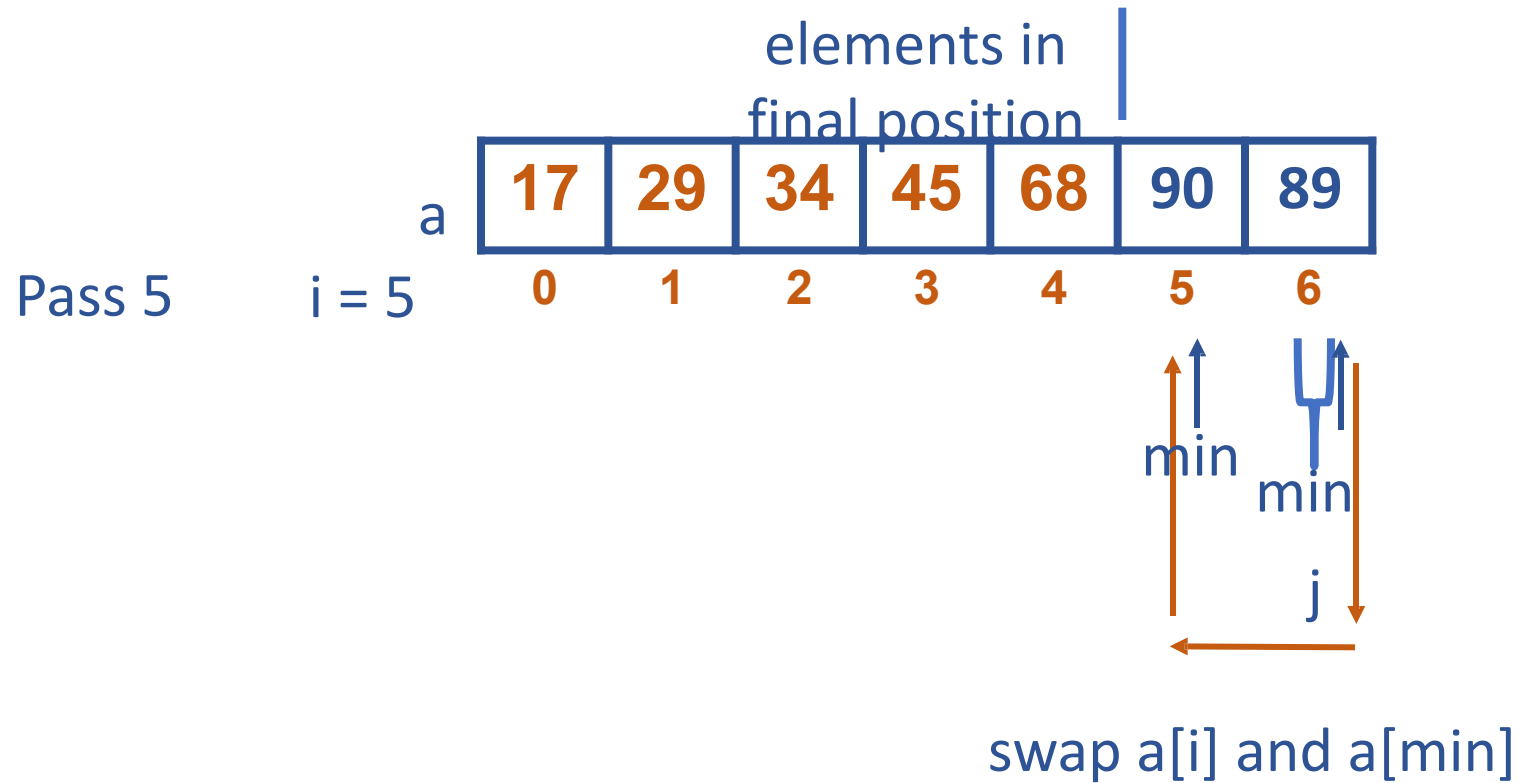
$\text{min} = 4$



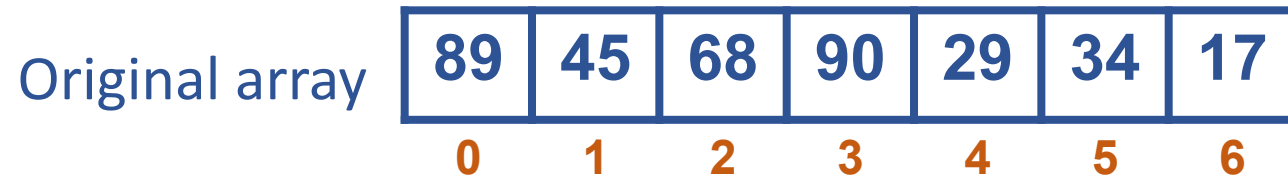
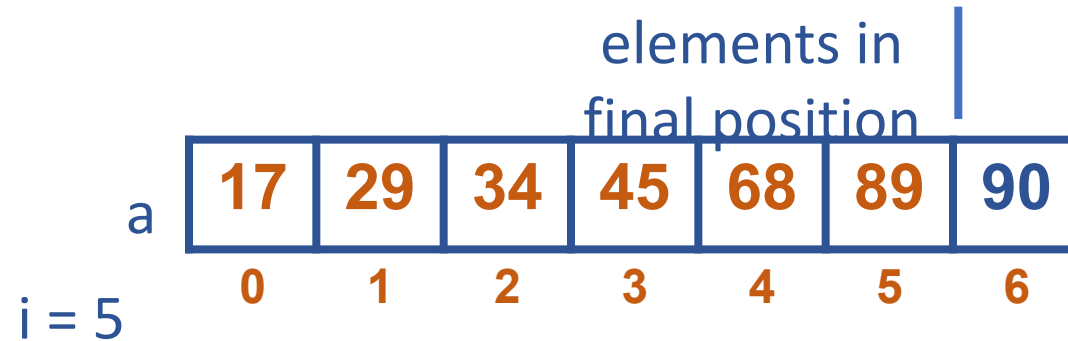


min = 5





$\text{min} = 6$



ALGORITHM SelectionSort($A[0 \dots n-1]$)

//Sorts a given array by selection sort

//Input: An array $A[0 \dots n-1]$ of orderable elements

//Output: Array $A[0 \dots n-1]$ sorted in ascending order

for $i \leftarrow 0$ to $n-2$ do

$\text{min} \leftarrow i$

 for $j \leftarrow i+1$ to $n-1$ do

 if $A[j] < A[\text{min}]$ $\text{min} \leftarrow j$

 swap $A[i]$ and $A[\text{min}]$

Analysis of Selection Sort:

Input Size: n (size of the list)

Basic Operation: Comparison ($A[j] < A[\min]$)

$$C_{\text{best}}(n) = C_{\text{worst}}(n) = C_{\text{avg}}(n)$$

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i)$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

- Compare adjacent elements of the list and exchange them if they are out of order
- By doing it repeatedly, we end up bubbling the largest element to the last position on the list
- The next pass bubbles up the second largest element and so on and after $n - 1$ passes, the list is sorted
- Pass i ($0 \leq i \leq n - 2$) can be represented as follows:

$A[0], A[1], A[2], \dots, A[j] \quad \longleftrightarrow \quad A[j+1], \dots, A[n-i-1] \mid A[n-i] \leq \dots \leq A[n-1]$

in their final positions

$i = 0$

89	45	68	90	29	34	17
0	1	2	3	4	5	6

j

elements in
final position

$j = 0$

4 ⁵ 8 ⁴ 9 ⁰	4 ⁸ 5 ¹ 9 ⁵	68 2	90 3	29 4	34 5	17 6
45	89					

☒ swap

$i = 0$

45	89	68	90	29	34	17
0	1	2	3	4	5	6

j

elements in
final position

$j = 1$

45	6 ₈	6 ₈ ⁸	90	29	34	17
0	8 ₉ ¹	9 ₂ ²	3	4	5	6

$68 < 89$ ☒ swap

$i = 0$

45	68	89	90	29	34	17
0	1	2	3	4	5	6

j

elements in
final position

$j = 2$

45	68	89	90	29	34	17
0	1	2	3	4	5	6

$90 < 89$  no swap

$i = 0$

45	68	89	90	29	34	17
0	1	2	3	4	5	6

j

elements in
final position

$j = 3$

45	68	89	29	29	34	17
0	1	2	3	4	5	6

$29 < 90$ ☒ swap

$i = 0$

45	68	89	29	90	34	17
0	1	2	3	4	5	6

j

elements in
final position

$j = 4$

45	68	89	29	90	34	17
0	1	2	3	4	5	6

$34 < 90$ ☒ swap

$i = 0$

45	68	89	29	34	90	17
0	1	2	3	4	5	6

j

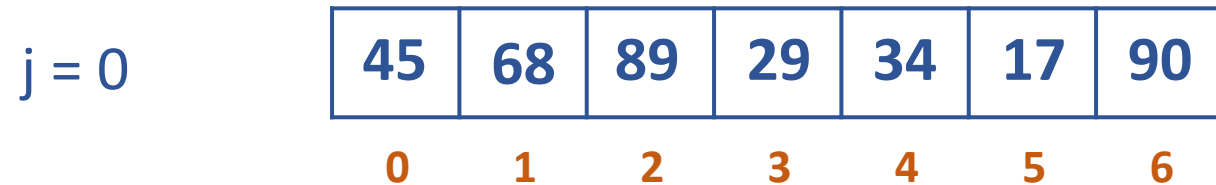
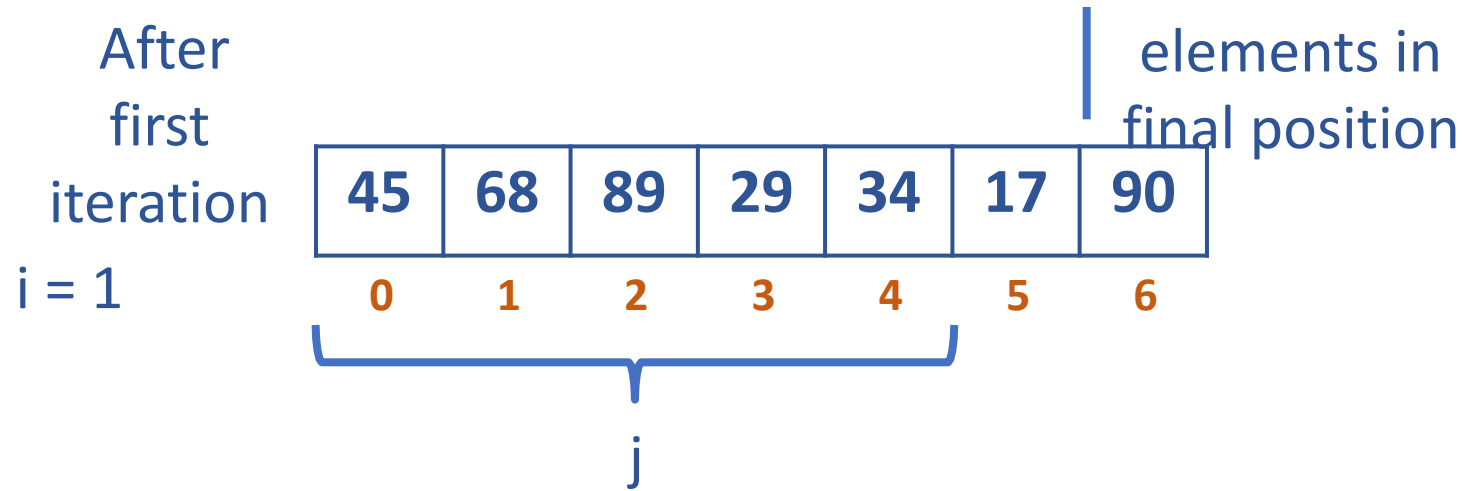
elements in
final position

$j = 5$

45	68	89	29	34	17	90
0	1	2	3	4	5	6

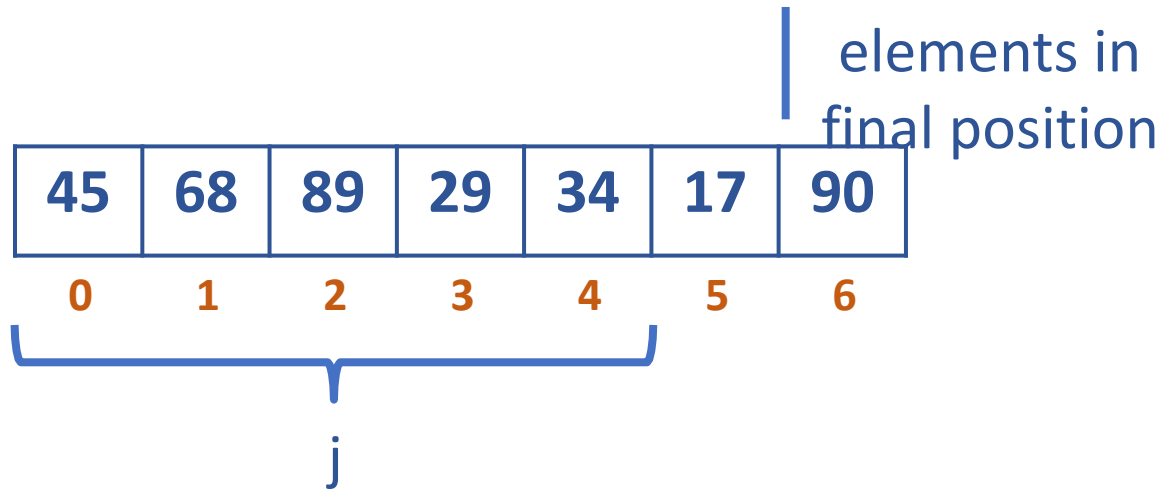
$17 < 90$ ☒ swap

Bubble Sort



$68 < 45$  no swap

$i = 1$

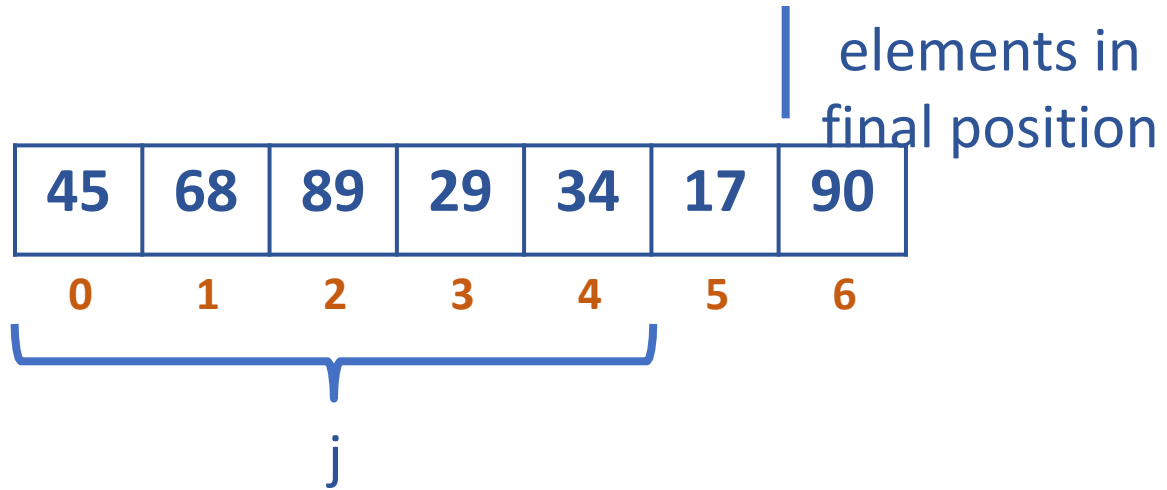


$j = 1$

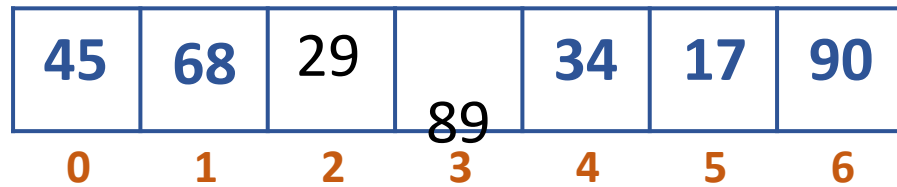
45	68	89	29	34	17	90
0	1	2	3	4	5	6

$89 < 68$  no swap

$i = 1$

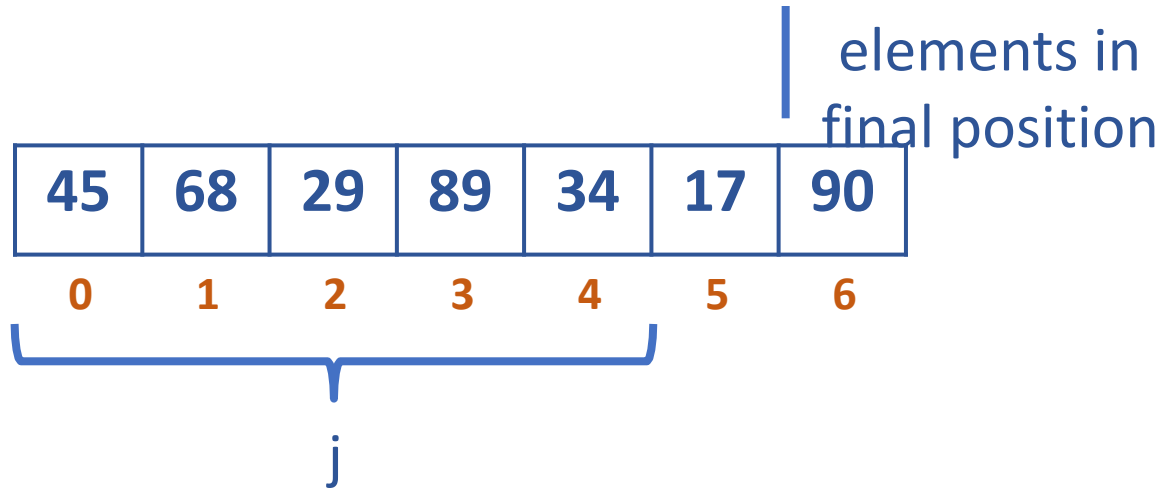


$j = 2$



$29 < 89$  swap

$i = 1$

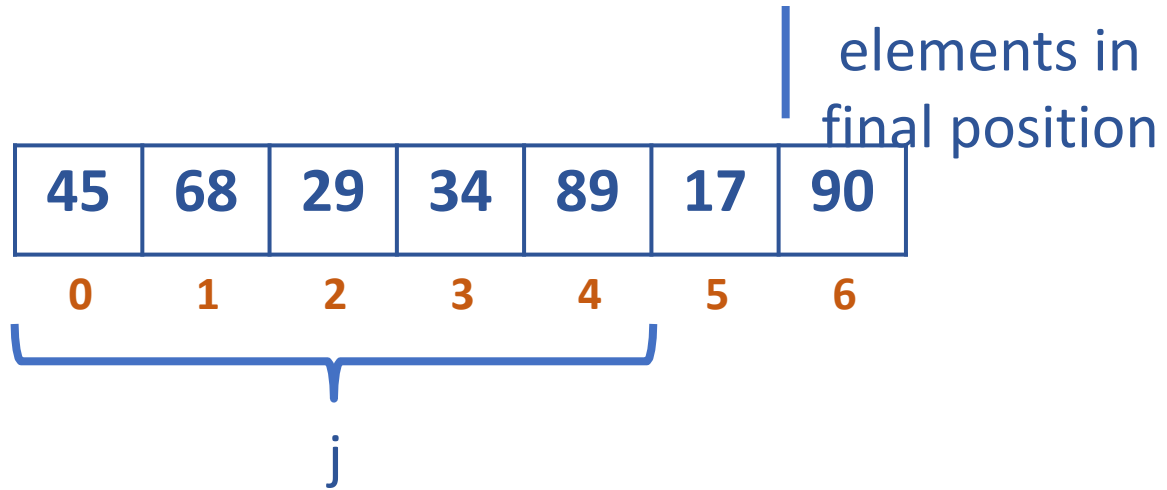


$j = 3$

45	68	29	34	89	17	90
0	1	2	3	4	5	6

$34 < 89$ ☒ swap

$i = 1$



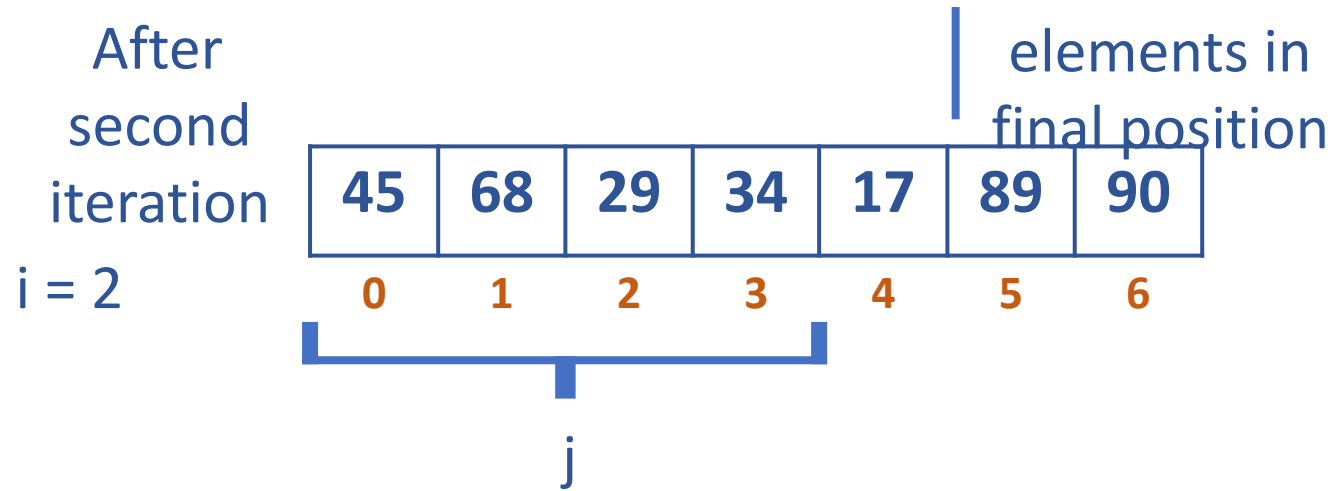
$j = 4$



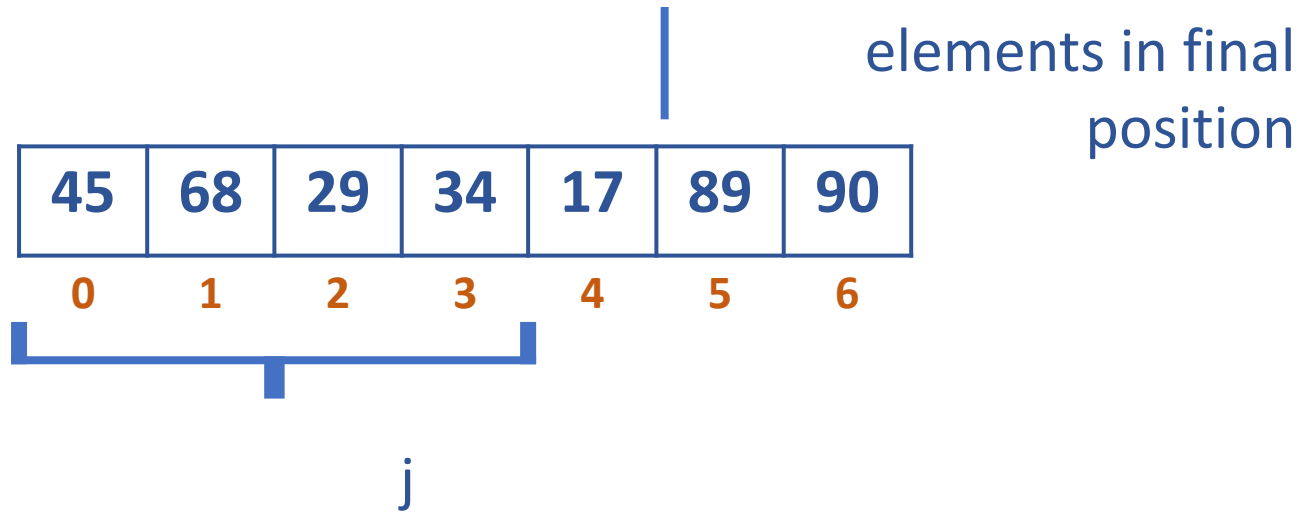
$17 < 89$ ☒ swap

DESIGN AND ANALYSIS OF ALGORITHMS

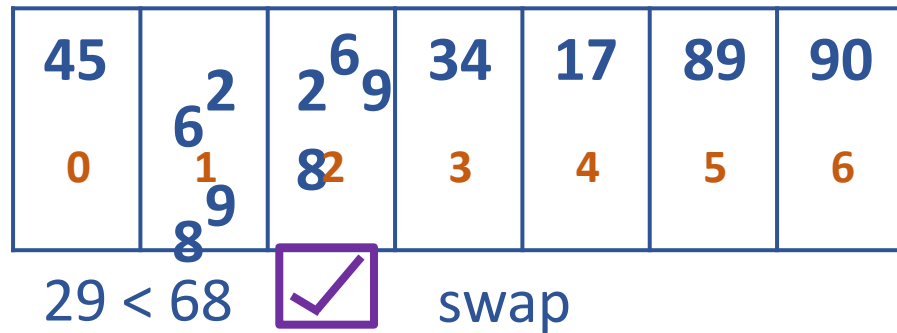
Bubble Sort



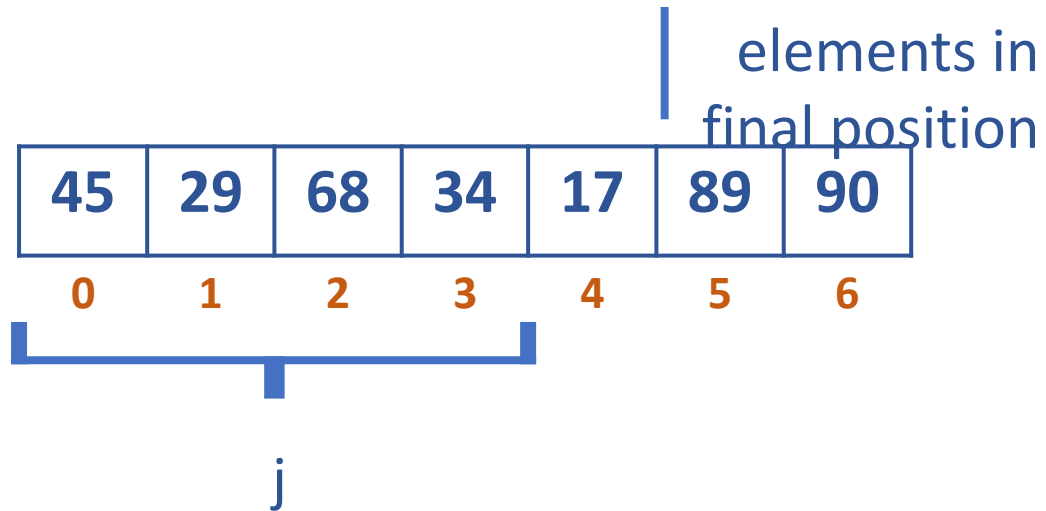
$68 < 45$  no swap



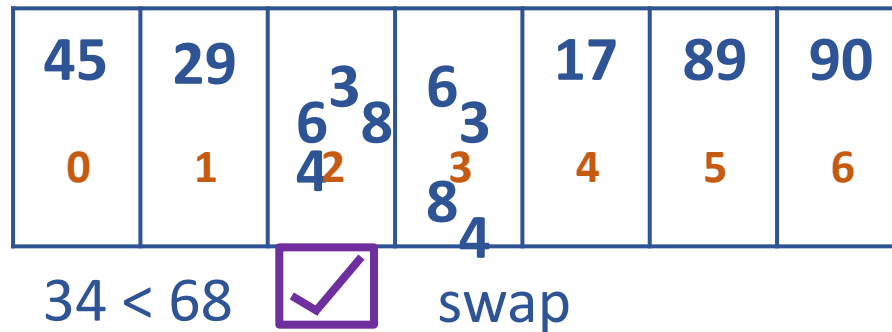
$j = 1$



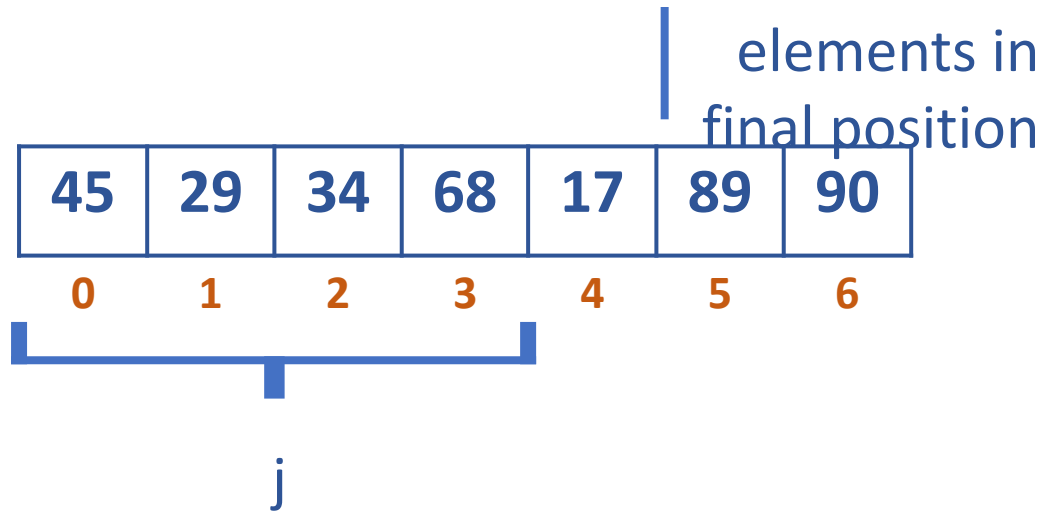
$i = 2$



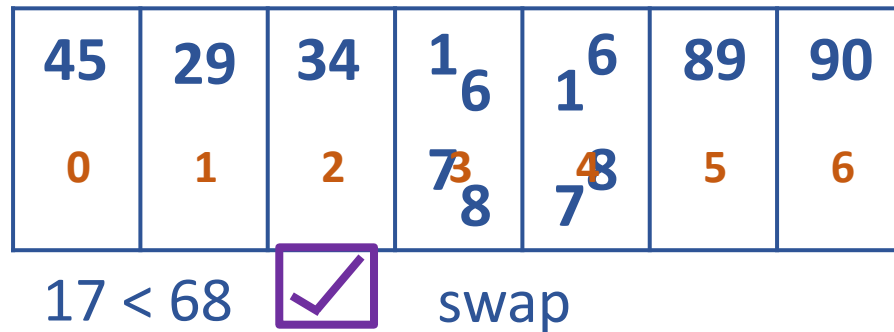
$j = 2$



$i = 2$

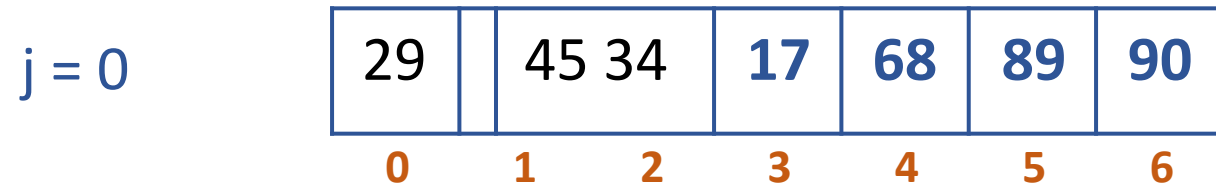
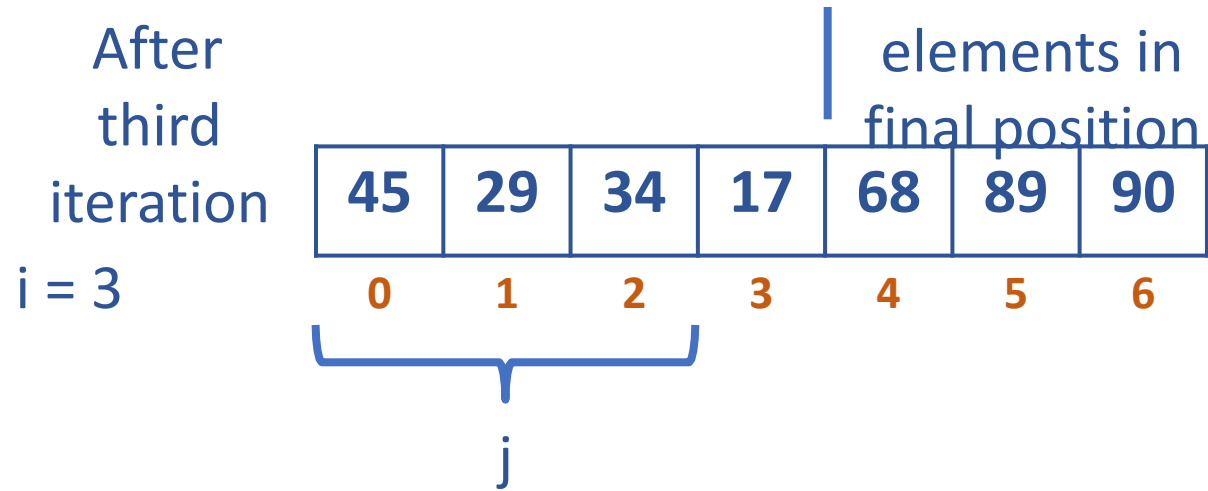


$j = 3$



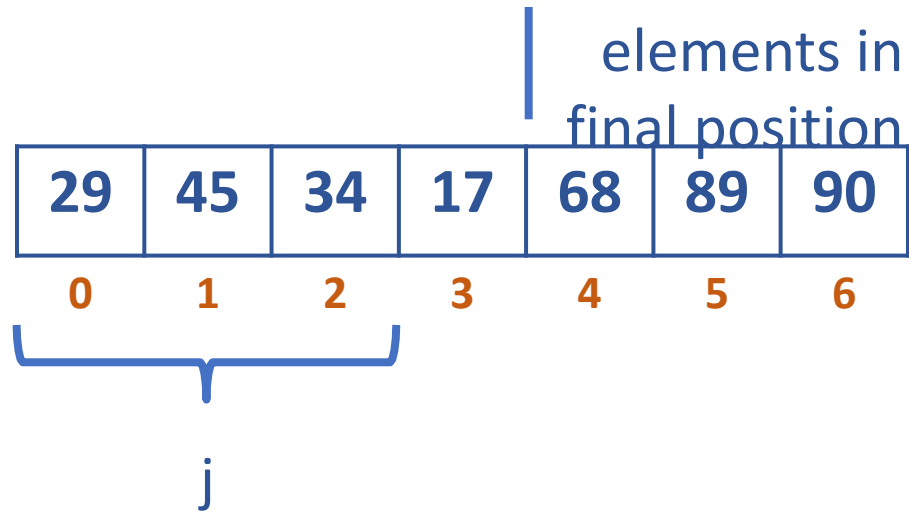
DESIGN AND ANALYSIS OF ALGORITHMS

Bubble Sort

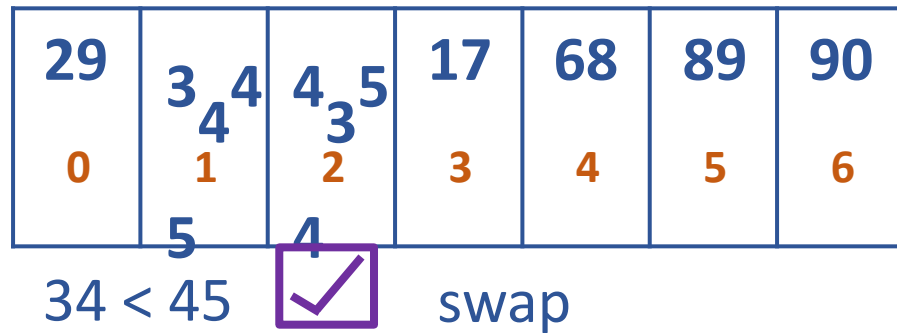


$29 < 45$  swap

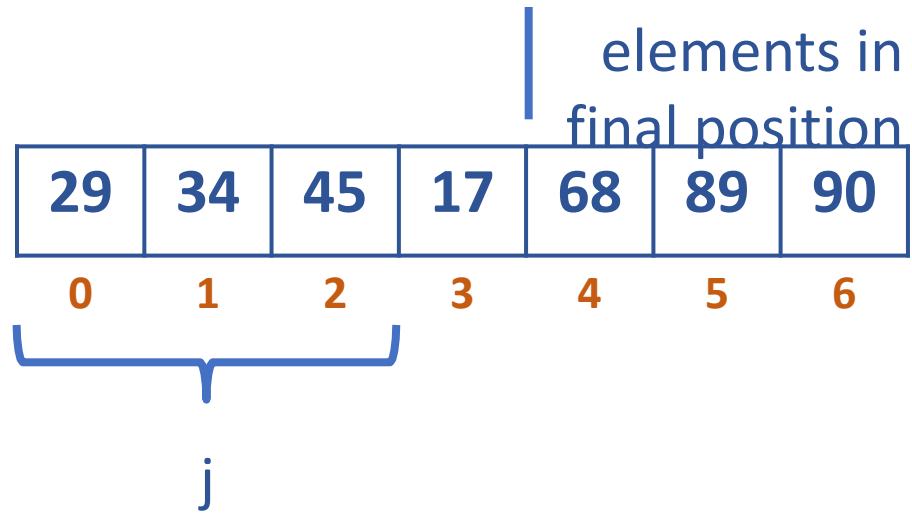
$i = 3$



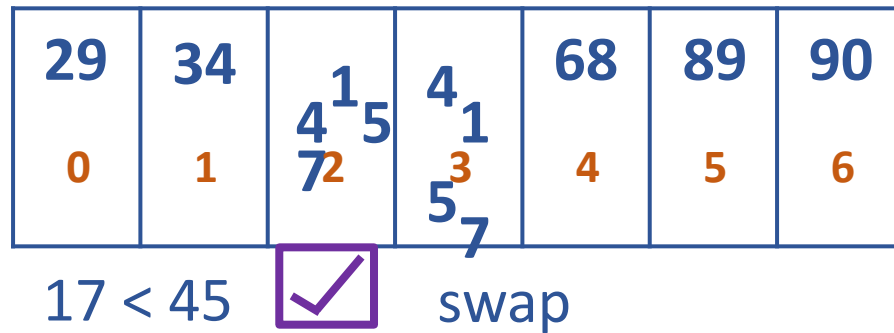
$j = 1$



$i = 3$

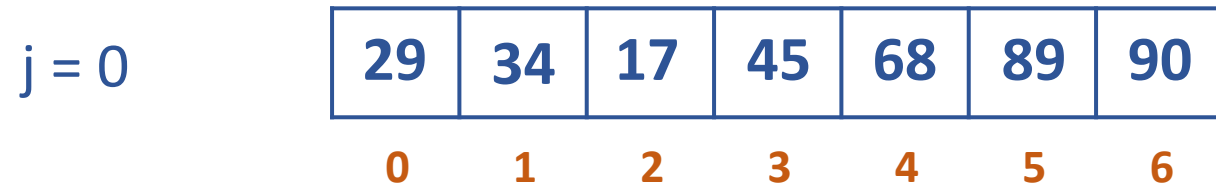
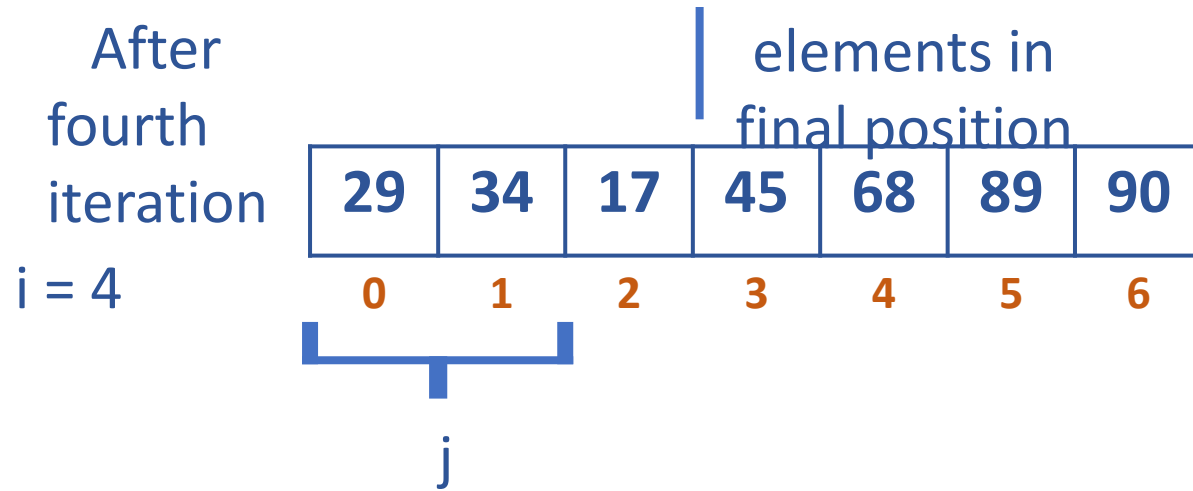


$j = 2$

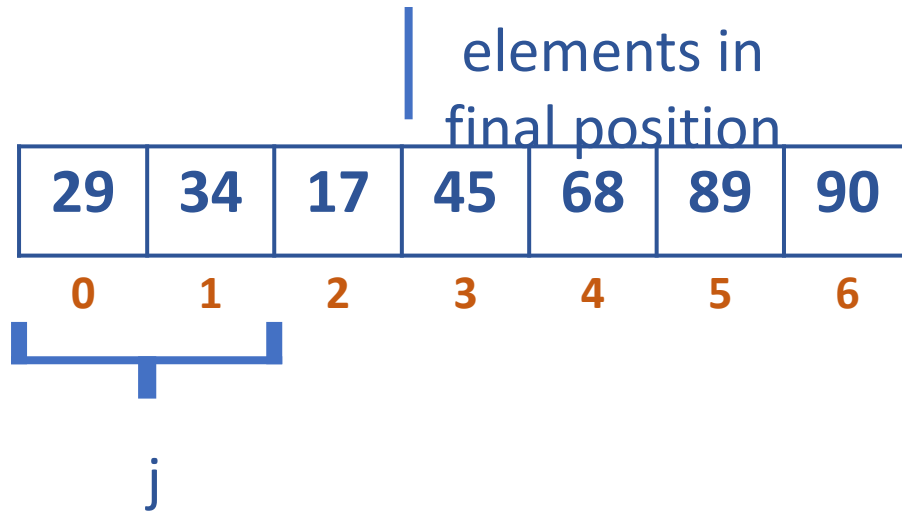


DESIGN AND ANALYSIS OF ALGORITHMS

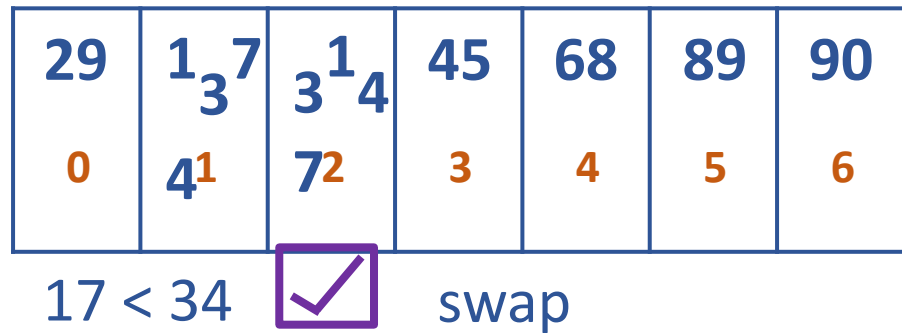
Bubble Sort



$34 < 29$  no swap

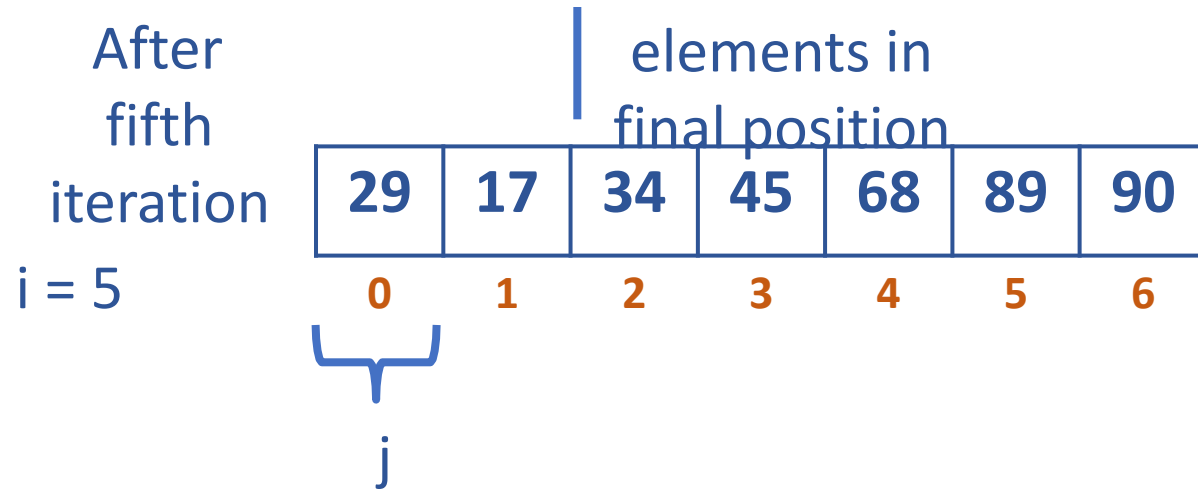


$j = 1$

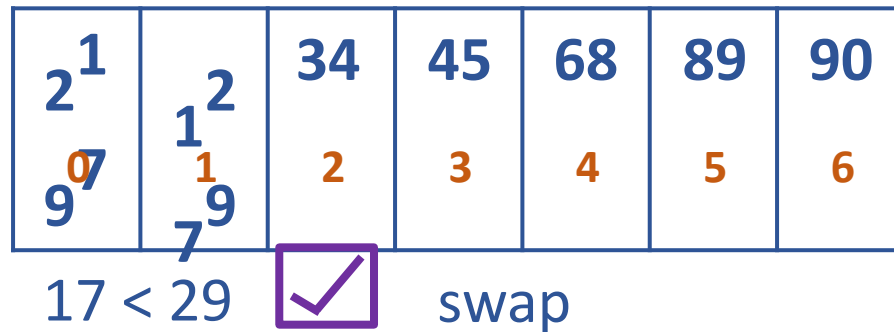


DESIGN AND ANALYSIS OF ALGORITHMS

Bubble Sort



$j = 0$



Bubble Sort

After sixth iteration

elements in final position							
17	29	34	45	68	89	90	
0	1	2	3	4	5	6	

ALGORITHM BubbleSort($A[0 \dots n - 1]$)

//Sorts a given array by bubble sort in their final positions

//Input: An array $A[0 \dots n - 1]$ of orderable elements

//Output: Array $A[0 \dots n - 1]$ sorted in ascending order

for $i \leftarrow 0$ to $n - 2$ do

 for $j \leftarrow 0$ to $n - 2 - i$ do

 if $A[j + 1] < A[j]$ swap $A[j]$ and $A[j + 1]$

Bubble Sort Analysis

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1]$$

$$= \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Bubble Sort is a $\Theta(n^2)$
algorithm

Bubble Sort (improved version):

Algorithm BubbleSortImproved($A[0..n-1]$)

//Sorts by an improved Bubble Sort algorithm.

//Input: An array **A** of orderable elements by $\leq$.

//Output: Sorted array A.

for $i \leftarrow 0$ **to** $n-2$ //n-1 passes

anySwaps $\leftarrow$ **FALSE**

for $j \leftarrow 0$ **to** $n-2-i$ //last i elements are sorted

if ($A[j] > A[j+1]$)

 Swap $A[j]$ with $A[j+1]$

anySwaps $\leftarrow$ **TRUE**

if (**anySwaps** = **FALSE**)

return

return

Algorithm BubbleSortImproved(A[0..n-1])

Input Size: n

Basic Operation : (a[j] > a[j+1])

$$C_{\text{worst}}(n) = n * (n - 1) / 2 \in \Theta(n^2)$$

$$C_{\text{best}}(n) = (n - 1) \in \Theta(n)$$

Basic Operation : Swap

$$C_{\text{worst}}(n) = n * (n - 1) / 2 \in \Theta(n^2)$$

$$C_{\text{best}}(n) = 0 \in \Theta(1)$$

Which of the following algorithms should NOT be considered an application of the brute-force approach?

- a) Linear search in an unsorted array
- b) Selection sort
- c) Euclid's algorithm for finding the GCD
- d) Naive string matching algorithm

Answer c) Euclid's algorithm for finding the GCD

Which of the following problems cannot be solved efficiently using a brute-force approach due to its exponential time complexity?

- a) Finding the maximum element in an array
- b) Traveling Salesperson Problem (TSP)
- c) Linear search in an unsorted list
- d) Matrix addition

Answer b) Traveling Salesperson Problem (TSP)

Which of the following statements about the time complexity of Selection Sort is correct?

- a) The best-case time complexity is $\Theta(n)$ because the array is already sorted
- b) The worst-case time complexity is $\Theta(n \log n)$ due to repeated swaps
- c) The best, average, and worst-case time complexities are all $\Theta(n^2)$ because the number of comparisons is independent of input order
- d) The average-case time complexity is $\Theta(n \log n)$, but the worst-case is $\Theta(n^2)$

Answer : The best, average, and worst-case time complexities are all $\Theta(n^2)$ because the number of comparisons is independent of input order

Which of the following statements correctly describes the stability property of Selection Sort?

- a) Selection Sort is stable because it always preserves the relative order of equal elements.
- b) Selection Sort is unstable because swapping the minimum element with the first unsorted element may change the relative order of equal keys
- c) Selection Sort becomes stable if comparisons are performed before swaps
- d) Selection Sort becomes stable if the number of swaps is minimized

Answer b) Selection Sort is unstable because swapping the minimum element with the first unsorted element may change the relative order of equal keys

Which of the following statements about the Bubble Sort algorithm is correct?

- a) Bubble Sort is unstable because swapping adjacent elements may change the relative order of equal keys
- b) Bubble Sort has a worst-case time complexity of $\Theta(n \log n)$ due to repeated passes.
- c) Bubble Sort is stable and adaptive; with an early-exit (swap-flag) optimization, its best-case time complexity is $\Theta(n)$
- d) Bubble Sort performs the same number of comparisons in best, average, and worst cases

Answer : c) Bubble Sort is stable and adaptive; with an early-exit (swap-flag) optimization, its best-case time complexity is $\Theta(n)$

Consider the optimized Bubble Sort algorithm that terminates early if no swaps occur in a pass. Which of the following statements is correct?

- a) The optimized Bubble Sort reduces the worst-case time complexity to $\Theta(n)$
- b) In the worst case, Bubble Sort performs exactly $n-1$ swaps
- c) In the best case, the optimized Bubble Sort completes in one pass with $n-1$ comparisons
- d) Bubble Sort is unstable because elements can move more than one position in a single pass

Answer : c) In the best case, the optimized Bubble Sort completes in one pass with $n-1$ comparisons

Which of the following statements correctly compares Selection Sort and Bubble Sort?

- a) Both Selection Sort and Bubble Sort are stable and adaptive sorting algorithms
- b) Selection Sort is stable, while Bubble Sort is unstable due to multiple swaps
- c) Selection Sort performs fewer swaps but is unstable, whereas Bubble Sort is stable and adaptive (with early-exit optimization)
- d) Bubble Sort always performs fewer comparisons than Selection Sort for all inputs

Answer : c) Selection Sort performs fewer swaps but is unstable, whereas Bubble Sort is stable and adaptive (with early-exit optimization)

Which of the following statements correctly describes the in-place property of Selection Sort and Bubble Sort?

- a) Selection Sort is in-place, while Bubble Sort requires additional auxiliary memory for swapping
- b) Both Selection Sort and Bubble Sort are in-place algorithms, requiring only a constant amount of extra space
- c) Bubble Sort is not in-place because elements may move multiple positions during swapping
- d) Selection Sort is not in-place because it uses a temporary variable to perform swaps

Answer : b) Both Selection Sort and Bubble Sort are in-place algorithms, requiring only a constant amount of extra space



THANK YOU

Shylaja S S

Department of Computer Science
& Engineering

shylaja.sharath@pes.edu